5. High-Level UML Diagrams

This section shows the SpatialOS design in UML and how much of it is built for M2. Section 5.1 gives the class diagrams: the data classes used in M2 (Figure 5.1), the live localization and health classes (Figure 5.2a), the classes planned for M3 to M5 (Figure 5.2b), and the code units that work on them (Figures 5.3a and 5.3b). Section 5.2 gives sequence diagrams for the seven major flows. Flows 1 to 6 follow the full system design; Flow 7 shows the live position on the route, which is part of the M2 demo. Section 5.3 maps each flow to its requirements, Section 5.4 lists the main design decisions, and Section 5.5 states which code exists today.

Names match Section 1 (tables, fields, enums) and the code. Scope tags also follow Section 1: **[M2]** is built for this milestone, **[M2 partial]** is partly built, and **[Planned]** is built in M3 to M5. A requirement scheduled for a later milestone that is already working is marked *built early in M2*. In the sequence diagrams, gray shaded blocks mark later-milestone steps, and each block is labeled with its milestone.

Coordinate frame in M2. The world frame is the VPS map frame of *SFSU HSS 349 v2*: right-handed, Y up, meters. The scanned mesh was exported from this map and the four graph nodes were recorded with the phone while localized on it, so the model, the graph and the live poses line up without a stored transform. The server only flips Z, because Unity is left-handed.

5.1 Class diagrams

Five figures follow, one per page. What to notice in each:

Figure 5.1:

- A Site has map versions. A MapVersion owns its graph nodes, path segments, destinations and test points, and linked records must share one map version and graph revision.
- A PathSegment has two separate links to GraphNode: start node and end node. Its direction field says whether it can also be walked from end to start.
- Route is an API response in M2 and a stored routes table from M3. Each RouteStep gives the travel direction explicitly (from_node_id to to_node_id).
- The HSS 349 map has 4 graph nodes, 4 path segments and 4 destinations: Door, Back Corner (Door Side), Back Corner (Far Side) and Front of Room.
- In M2, LocalizationAttempt rows are labeled sample records used for list and search. The phone starts logging attempts in M3.
- In M2 the 3D model is a static GLB file served by Vercel (`public/models/hss349-v2.glb`). It moves to Supabase Storage and the visual_assets table in M3.

Figure 5.2a:

- PoseSample is built in M2. The pose_samples table stores every pose the phone sends through POST /api/poses, with the raw provider_pose and the converted world_position. Poses from the fake-test device are stored with data_origin = synthetic.
- From M3, a PoseSample belongs to a NavigationSession and records the transform used to convert it, and a successful live attempt links its pose through valid_pose_sample_id.
- FrameTransform is not needed for HSS 349 because the VPS frame is the world frame (Section 5 introduction). It becomes a table in M3, when other maps are added, and live navigation then uses only verified transforms.
- MetricSnapshot stores the exact attempt_ids it was calculated from, and an AiSummary explains one snapshot.

Figure 5.2b:

- SimulatedObstacle, AgentReport and PathEvent form the blockage chain. A tester places an obstacle, the robot reports it, and only an operator's confirmation closes the segment, through exactly one PathEvent.
- Every PathEvent records who made the change. Roles are per site through SiteMembership.

Figure 5.3a:

- Clients write only through API route handlers, which validate every request. The only direct database access from a browser is read only: /live reads the latest pose and subscribes to new poses through Supabase Realtime. Row-level security allows reads only; in M2 the pose policy still allows reading every pose, and limiting it to public demo maps is planned for M3.
- RoutePlanner (/viewer), LivePage (/live) and, from M3, the robot simulation all call RouteService, so a person, a live walker and a robot get routes from the same algorithm.
- The iPhone app sends its pose every 0.5 s through PoseUploader. Attempt logging (LocalizationClient) follows in M3.
- M2 has no sign-in. POST /api/poses is protected by a shared device token, and the read endpoints serve only the public demo map. Site roles arrive with RoleGuard in M3.

Figure 5.3b:

- Route handlers do input and output only. Request parsing is in validation, database access is in SupabaseRepository, and routing is in the pure RoutingEngine, so each part can be tested on its own.
- RouteService loads the graph, calls findRoute through the routing adapter, and checks every returned step against the database graph (validateRoutingResult) before answering. A wrong result is never sent to a viewer.
- PoseService decides data_origin on the server with poseOrigin: poses from the fake-test device, or marked synthetic by the client, are stored as synthetic. Poses from any other device are stored as real, so test builds must use the fake-test device or send data_origin = synthetic.
- HealthService will load attempts through SupabaseRepository, pass them to HealthCalculator and save the snapshot; the calculator never touches the database. RoleGuard will be the single place where site roles are checked.

Figure 5.1: M2 core data classes (selected core attributes)

The classes behind the M2 prototype: the scanned site, its graph, destinations, test points, sample attempts and the route response.

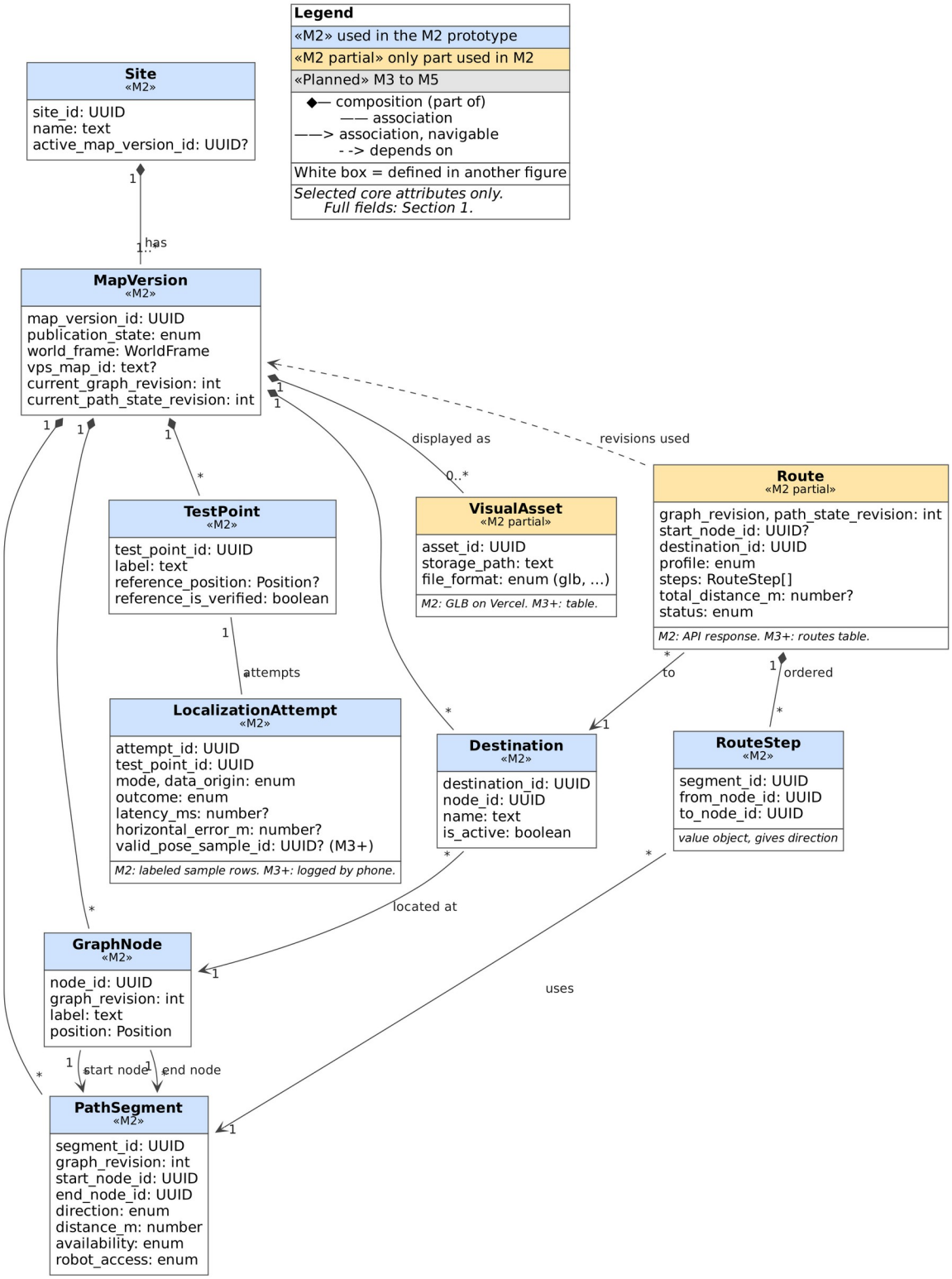


Figure 5.1: M2 core data classes (selected core attributes)

Figure 5.2a: Live localization and health data classes (M2 partial to M4)

Raw and converted poses, sessions, coordinate transforms and the health snapshot. White boxes are defined in Figure 5.1.

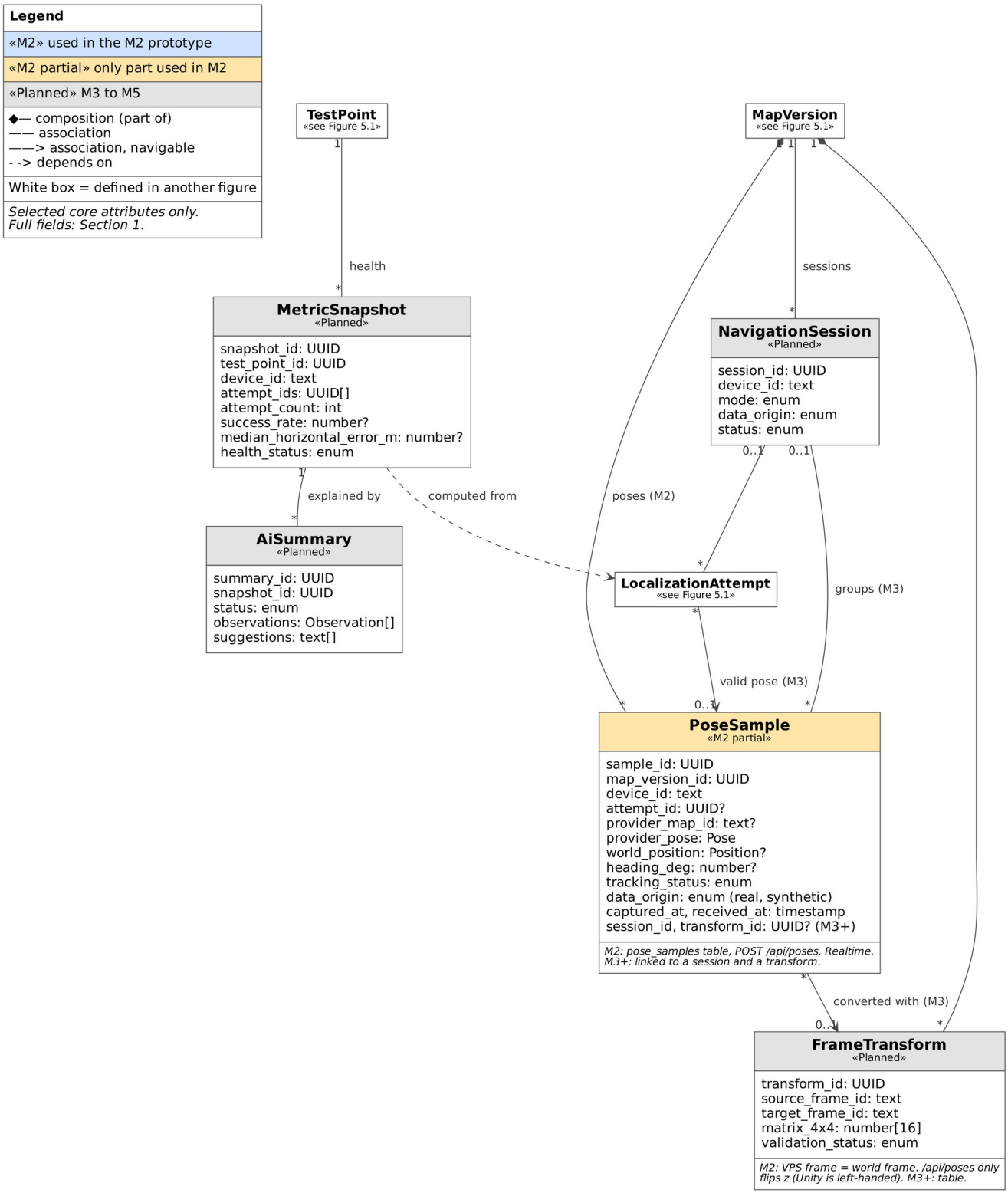


Figure 5.2a: Live localization and health data classes (M2 partial to M4)

Figure 5.2b: Planned data classes: simulation, path history and roles (M3 to M5)

The simulated robot, obstacles, operator review, path history and site roles.

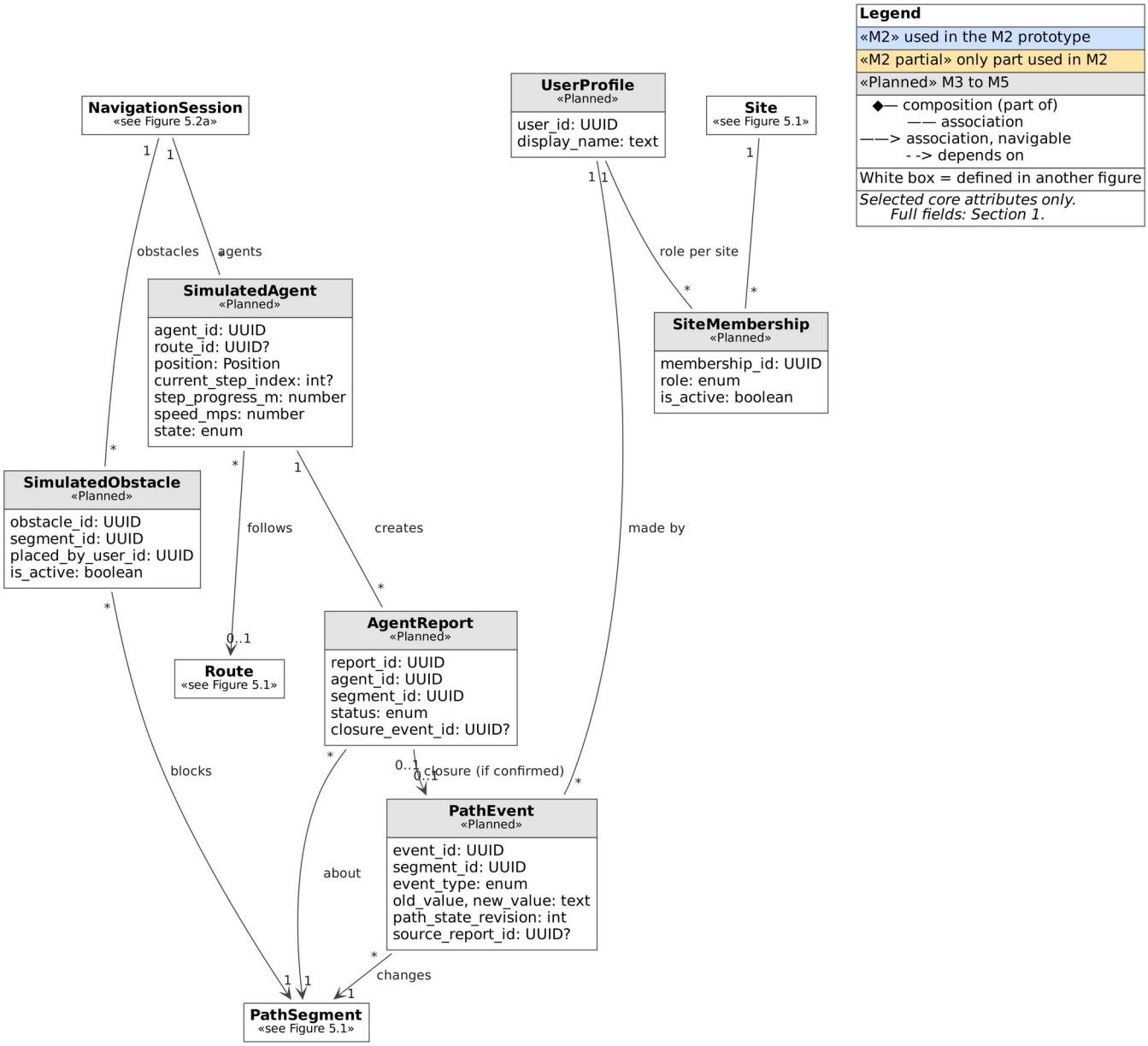


Figure 5.2b: Planned data classes: simulation, path history and roles (M3 to M5)

Figure 5.3a: Implementation units: clients and the API endpoints they call

Each box is tagged with its kind of unit (class, function module, React component, API route handler or Unity script) and whether it exists in the repository or is proposed.

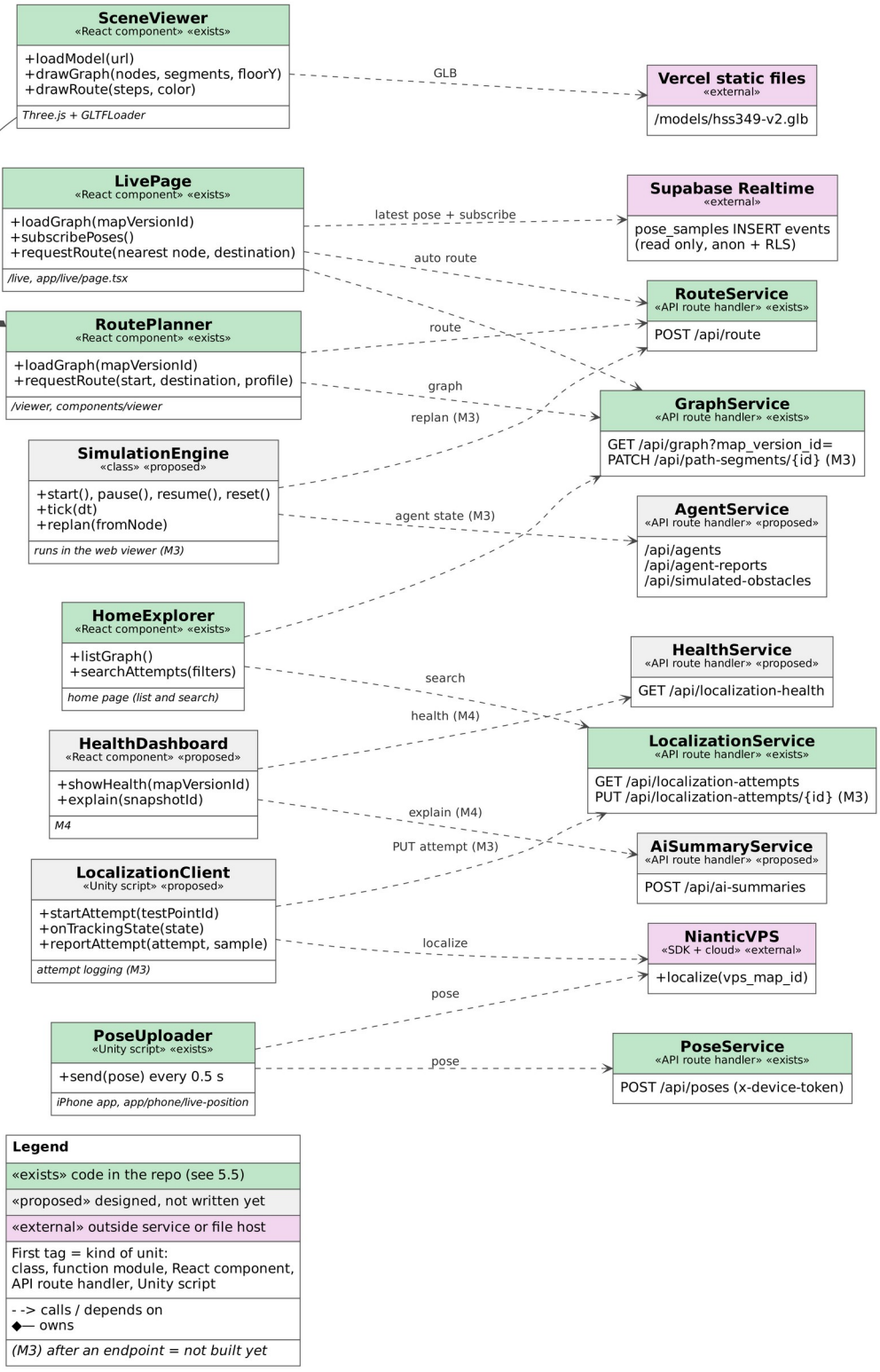


Figure 5.3a: Implementation units: clients and the API endpoints they call

Figure 5.3b: Implementation units: API handlers and core logic

What each API route handler does and what it depends on.

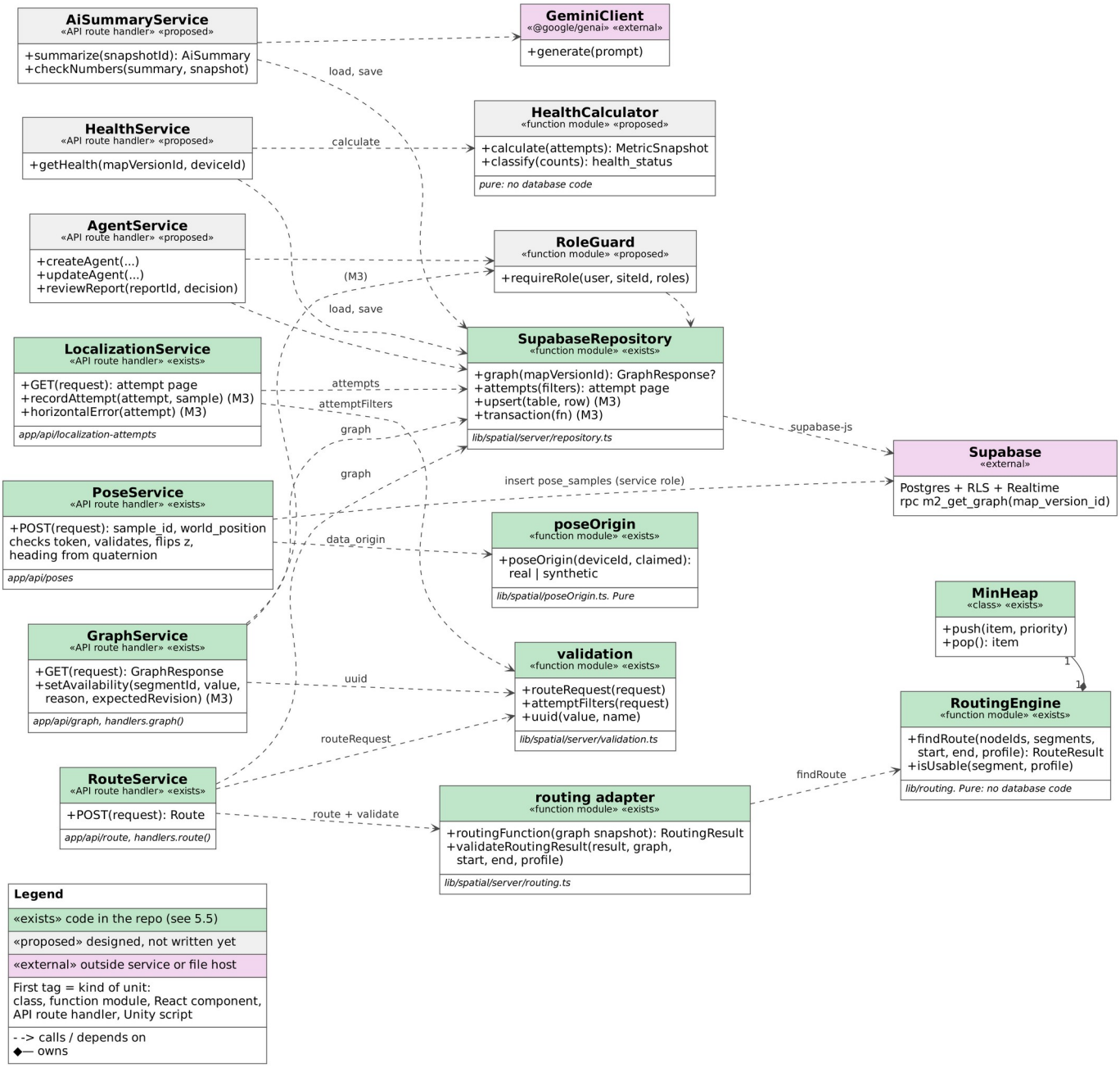


Figure 5.3b: Implementation units: API handlers and core logic

5.2 Sequence diagrams

Each flow lists its requirements and what is built in M2. Gray shaded blocks inside a diagram are later-milestone steps.

Flow 1: Localize the iPhone with VPS and show its status

Purpose: A developer stands at a test point and the iPhone localizes against the Niantic VPS map of the site.

Requirements: R3.1 (M2); R3.4, R4.1, R12.1, R12.2 (M3). Pose upload (R3.2) and the stale rule (R4.2) are built early in M2 and shown in Flow 7.

Scope: [M2 partial] M2: localization on the phone and its result (R3.1). M3: attempt records, timeout record and tracking states from the phone.

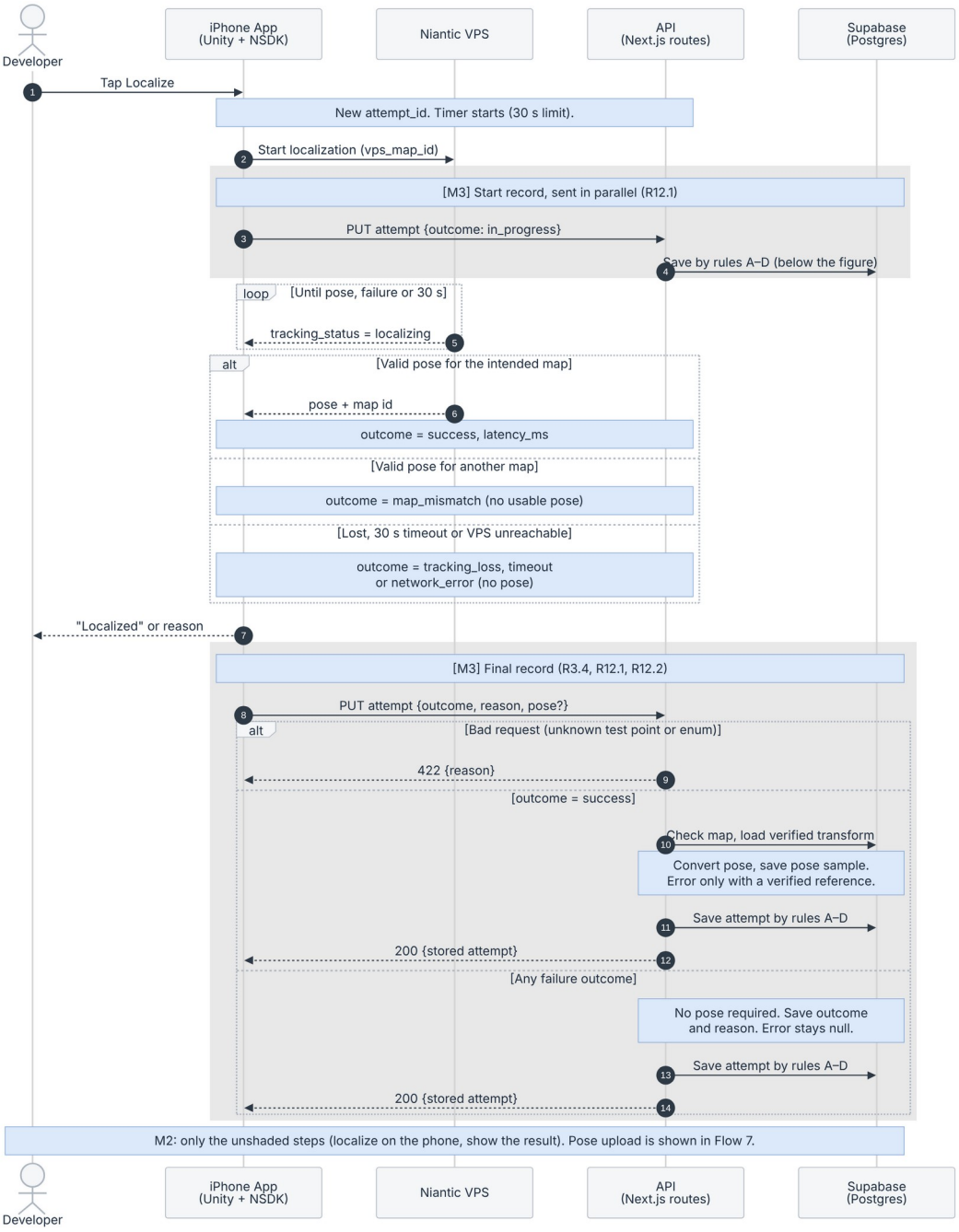


Figure 5.4: Localize the iPhone with VPS

What to notice

- The timer runs from the start of localization to the first valid pose. From M3 the start record is sent in parallel, so it never delays localization.
- Success means a valid pose for the intended map within 30 seconds. The SDK also reports its own tracking state (for example PRECISE); that is the SDK's confidence label, not a measured accuracy. Accuracy is measured separately at verified test points.
- Failures are always saved. map_mismatch, tracking_loss, timeout and network_error need no usable pose; the server stores the outcome and the reason, and horizontal error stays null (R12.2).
- Successes are checked first: the test point belongs to the map version, the VPS map matches its vps_map_id, and a verified transform exists. The phone sends the raw pose; the server converts it and calculates horizontal error only when the test point has a verified reference. Clients therefore cannot choose the stored error value.
- A pose older than 5 seconds, or with tracking that is not valid, cannot start a new live route (R4.2, R6.4, M3). In M2, /live already shows such a position as "Not live".

Save rules for an attempt (one atomic upsert on attempt_id)

A. No record exists: create it with the outcome received, finished or in_progress.

B. Stored outcome is in_progress and a finished outcome arrives: finish it.

C. The same finished outcome arrives again (a retry): change nothing and return the stored attempt.

D. An in_progress message arrives after the attempt is finished: change nothing. A finished attempt never goes back to in_progress.

The result is kept even if the start message is lost or arrives late, and retries never create duplicates. Pose samples are deduplicated by sample_id: resending the same sample does not create another record.

Flow 2: View the scanned area with its graph and four destinations

Purpose: A visitor opens /viewer and sees the scanned 3D model of HSS 349 with the route graph and the four labeled destinations.

Requirements: R2.2, R2.3, R2.4, R15.3 (M2). HSS 349 has four destinations, one per node.

Scope: [M2] Fully in M2. From M3 the active map comes from the site record and the model location from the visual_assets table.

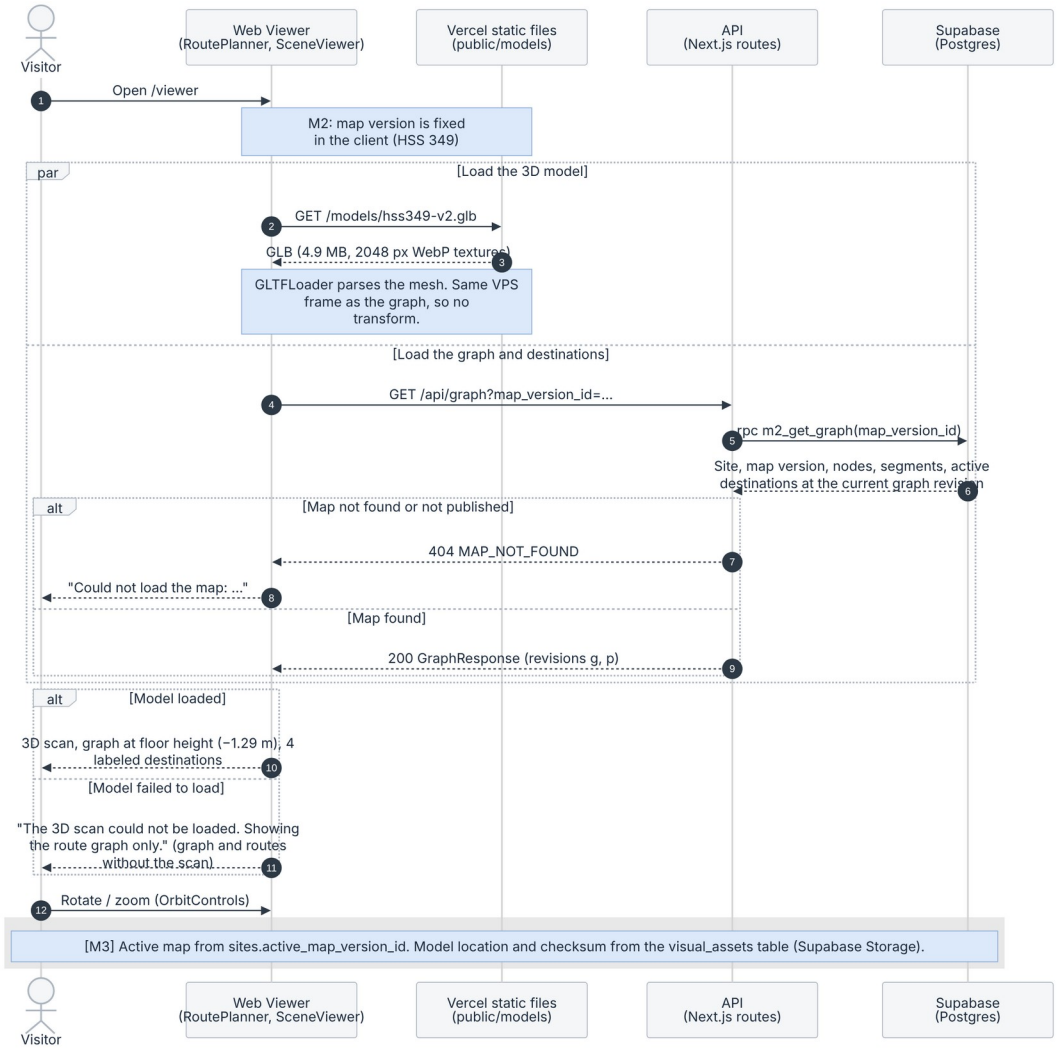


Figure 5.5: Load the scanned area, graph and destinations

What to notice

- The viewer loads the GLB from Vercel and the graph from GET /api/graph at the same time. In M2 the map version is fixed in the client; from M3 it is read from sites.active_map_version_id.
- The graph comes from one database function, m2_get_graph. It returns only a published, public map version, the rows of its current graph revision, and active destinations.
- The mesh shares the frame of the graph nodes, so no transform is applied. Nodes were recorded at phone height (y = 0), so the viewer draws the graph at the measured floor height of −1.29 m.
- If the model fails to load, the page shows an error message and still draws the graph, destinations and routes without the scan, so routing can still be shown. (A grid floor is drawn only when no model is configured.)
- Textures were reduced from 3072 to 2048 px WebP (14.5 MB to 4.9 MB) so the page loads quickly and stays within phone GPU memory.

Flow 3: Choose a start and destination, compute and draw the route

Purpose: A visitor picks a start and a destination by hand, chooses Person or Robot, and the route is drawn on the model.

Requirements: R5.1, R5.2, R5.3, R6.1 (M2); R5.4 robot route (M3) and R5.5 route distance (M4), both built early in M2.

Scope: [M2] Manual-start route for a person or the simulated robot, with its distance. M3: revision check on route requests and stored routes.

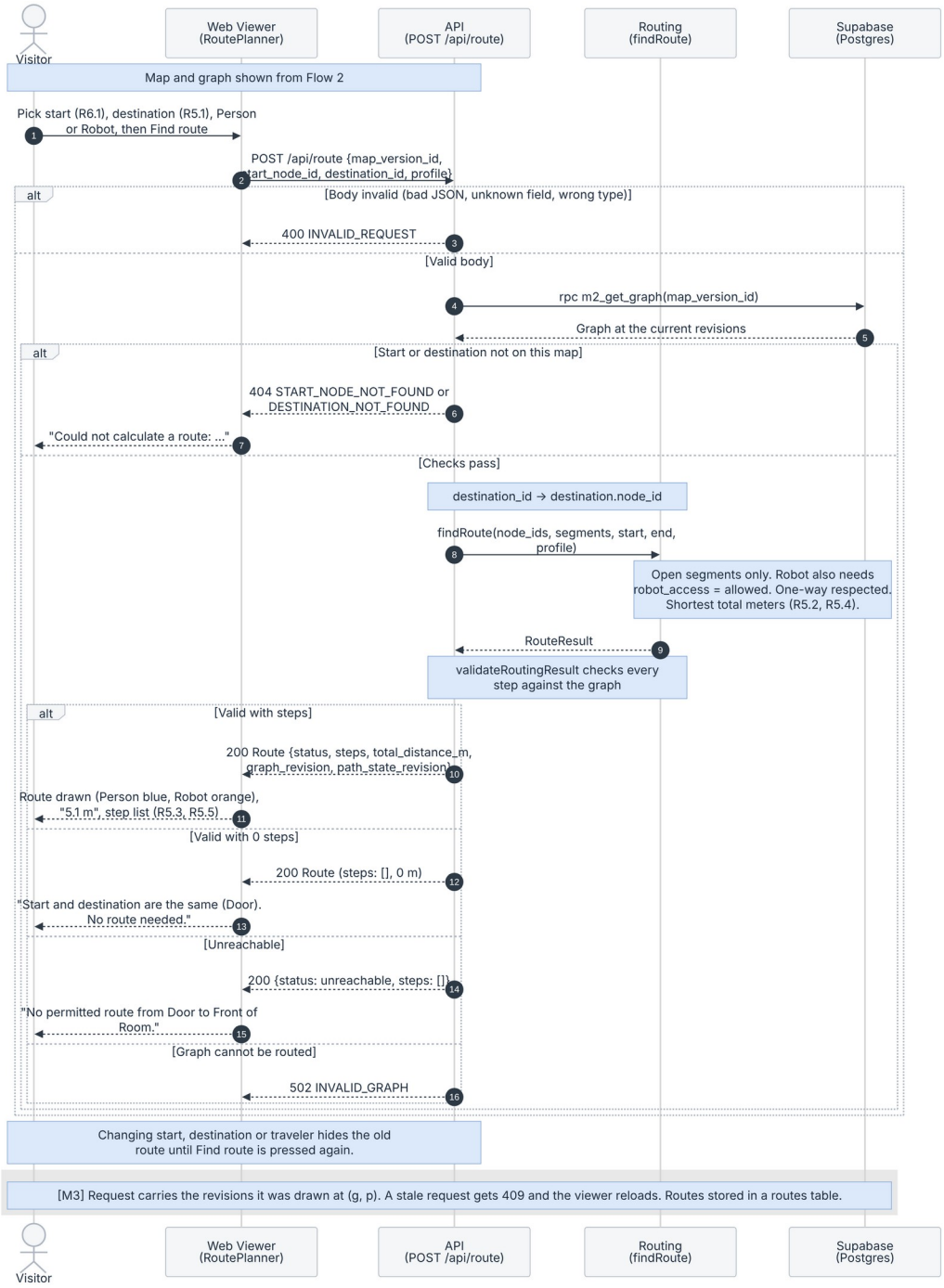


Figure 5.6: Route request

What to notice

- The API checks the request body first (400), then that the start node and destination belong to the map (404), then resolves destination_id to the destination's node before calling findRoute.
- findRoute uses open segments only, walks start_to_end segments in one direction only, and returns the shortest total distance (Dijkstra). For the robot it also skips segments with robot_access = restricted.
- validateRoutingResult checks every step against the database graph: connected, open, allowed for the profile, and no repeated node. A wrong result returns 502 and is never drawn.
- Results: valid (route drawn with its distance); valid with zero steps when start and destination are the same ("No route needed", 0 m); unreachable ("No permitted route"). invalid_input is an API error and is never stored as a Route.status.
- Changing the start, destination or traveler hides the old route until Find route is pressed again, so a person's route is never shown as the robot's.
- Distances are shown with one decimal (5.1 m) on /viewer and /live; the database keeps millimeter values. The response already carries graph_revision and path_state_revision; from M3 the request also sends the revisions it was drawn at, and a stale request gets 409 (decision 2).

Flow 4: Close or reopen a path segment, viewers update and reroute

Purpose: An operator closes a segment (for example, a wet floor) and every open route that used it is replanned.

Requirements: R8.1, R8.2, R8.3, R9.1, R9.3 (M3).

Scope: [Planned] M3. In M2, availability is seeded and can only be changed by hand in Supabase.

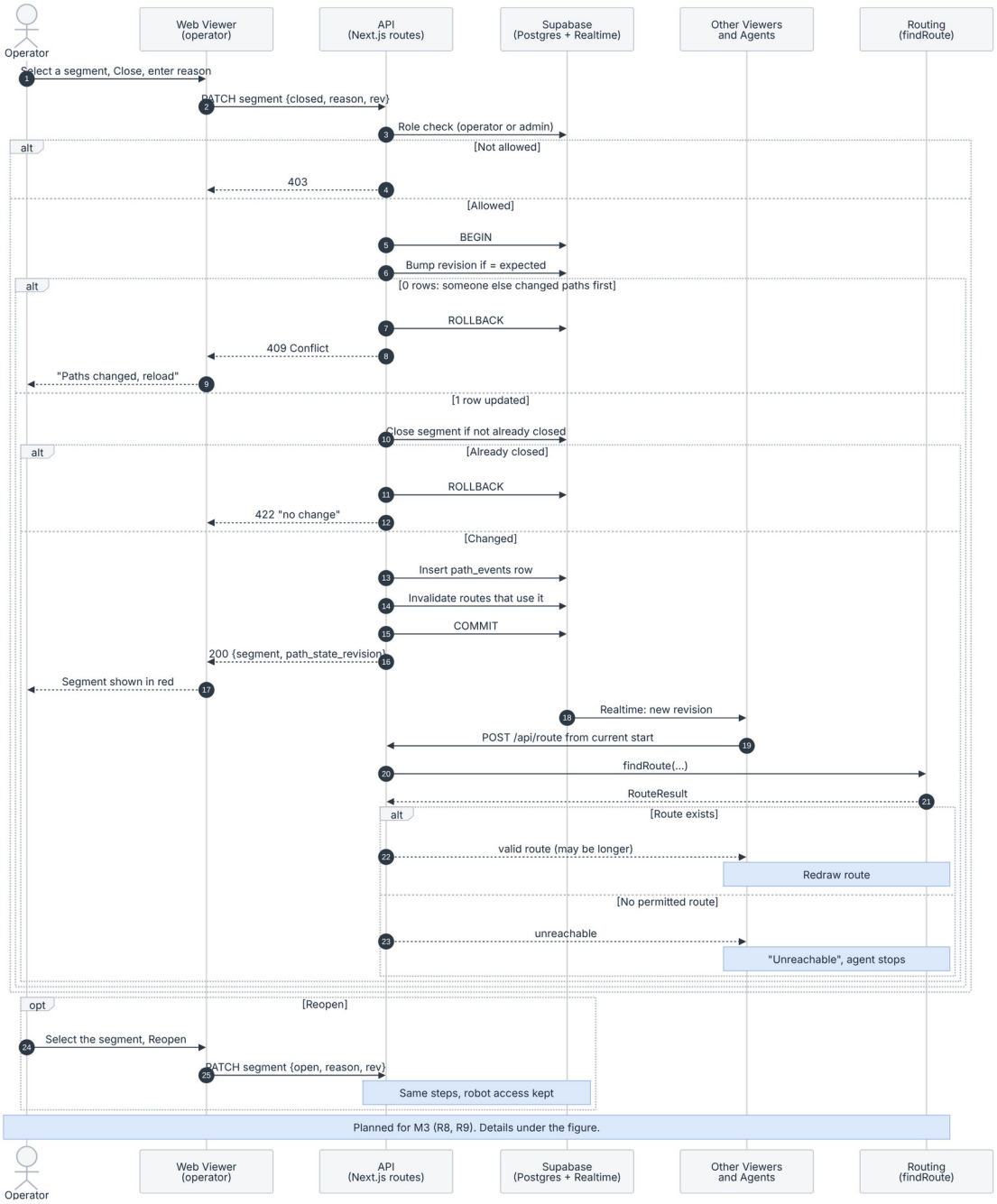


Figure 5.7: Path closure and rerouting

What to notice

- Step 6 is one conditional update: `UPDATE map_versions SET current_path_state_revision = expected + 1 WHERE map_version_id = … AND current_path_state_revision = expected`. If no row changes, another operator was first; the transaction rolls back and the client gets 409 Conflict.
- Step 10 changes the segment only if it is not already closed; otherwise the request gets 422 “no change”.

- The revision bump, the segment change, the path_events row and the invalidation of stored routes that use the segment commit in one transaction, so history never disagrees with the current state.
- Viewers and agents replan from their current routing start: a viewer from its chosen start node, a moving robot from its replan node (Figure 5.9). Reopening uses the same steps and keeps the segment's robot access (R17.2).

Flow 5: Run the simulated robot on allowed segments

Purpose: A tester starts a robot labeled SIMULATED. It follows a robot route, reacts to path changes and reports blockages to an operator. Figure 5.8 shows the run loop and controls, and Figure 5.9 shows the events checked on every tick.

Requirements: R10.1, R10.2, R10.3, R9.2 (M3); R10.4, R10.5, R18.1–R18.5 (M4); R18.6 (M5). The robot route profile itself (R5.4) is already built (Flow 3).

Scope: [Planned] M3: start, move and replan. M4: pause, reset, obstacles and operator confirmation. M5: operator rejection.

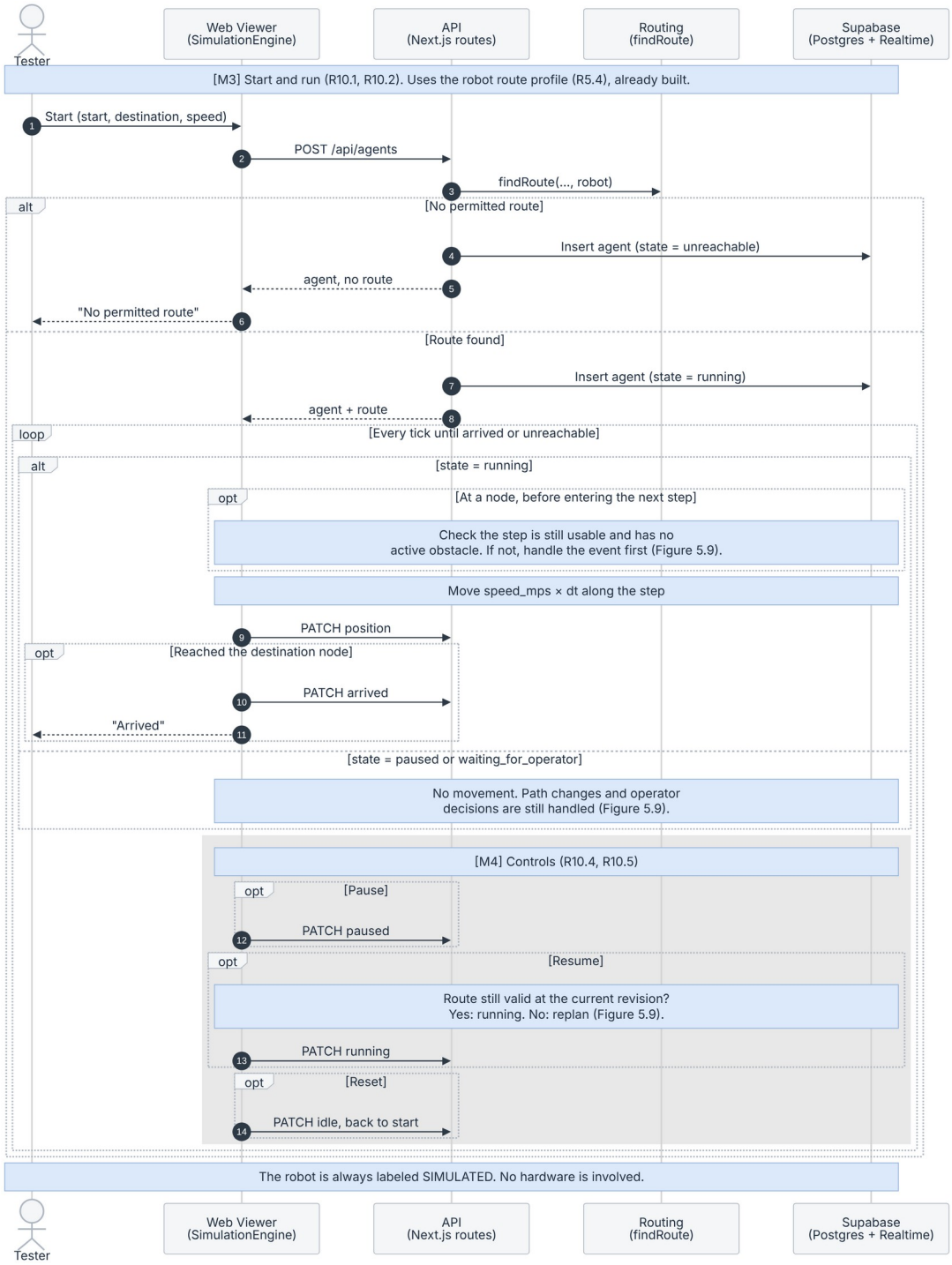


Figure 5.8: Simulated robot, run loop and controls

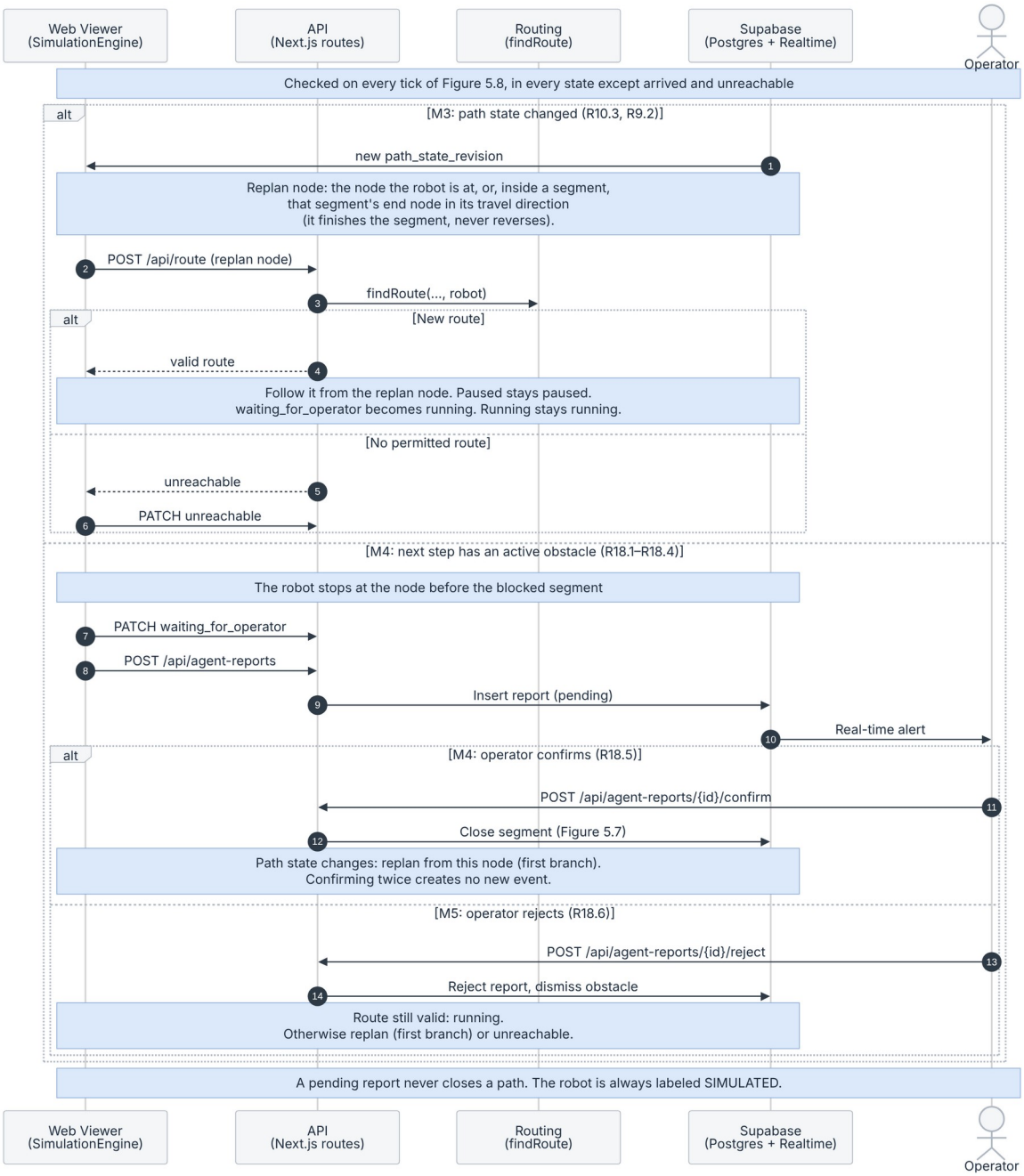


Figure 5.9: Simulated robot, path-change and obstacle events

What to notice

- Before entering each step, at the node, the robot checks that the step is still usable and has no active obstacle. Closures and obstacles are therefore caught before the robot moves into the affected segment.
- Events are handled in every state except arrived and unreachable. After a successful replan, a paused robot stays paused, a robot waiting for the operator's decision becomes running, and a running robot keeps running. No replacement route means unreachable.
- The replan starts at the node the robot is at. If the robot is inside a segment when it closes, it finishes that segment and replans from the segment's end node in its travel direction. It never reverses, so a one-way segment is never walked backwards, and it never skips a blocked segment.
- Pause, Resume and Reset are separate (M4). Resume first checks that the route is still valid at the current path_state_revision. Reset returns the robot to idle at its start with no route.

- Confirm (M4) closes the segment through Flow 4 and the robot replans. Reject (M5) dismisses the obstacle, and the robot resumes only if its route is still valid; otherwise it replans or becomes unreachable. The robot ends in arrived at the destination node.

Flow 6: Record a localization attempt and view test-point health

Purpose: A developer searches recorded attempts and, from M4, sees a health color for each test point.

Requirements: R15.1 (M2); R12.1, R12.2 (M3); R14.1–R14.5, R16.1 (M4).

Scope: [M2 partial] M2: search of labeled sample attempts. M3: attempts recorded by the phone (Flow 1). M4: health markers and the Gemini summary.

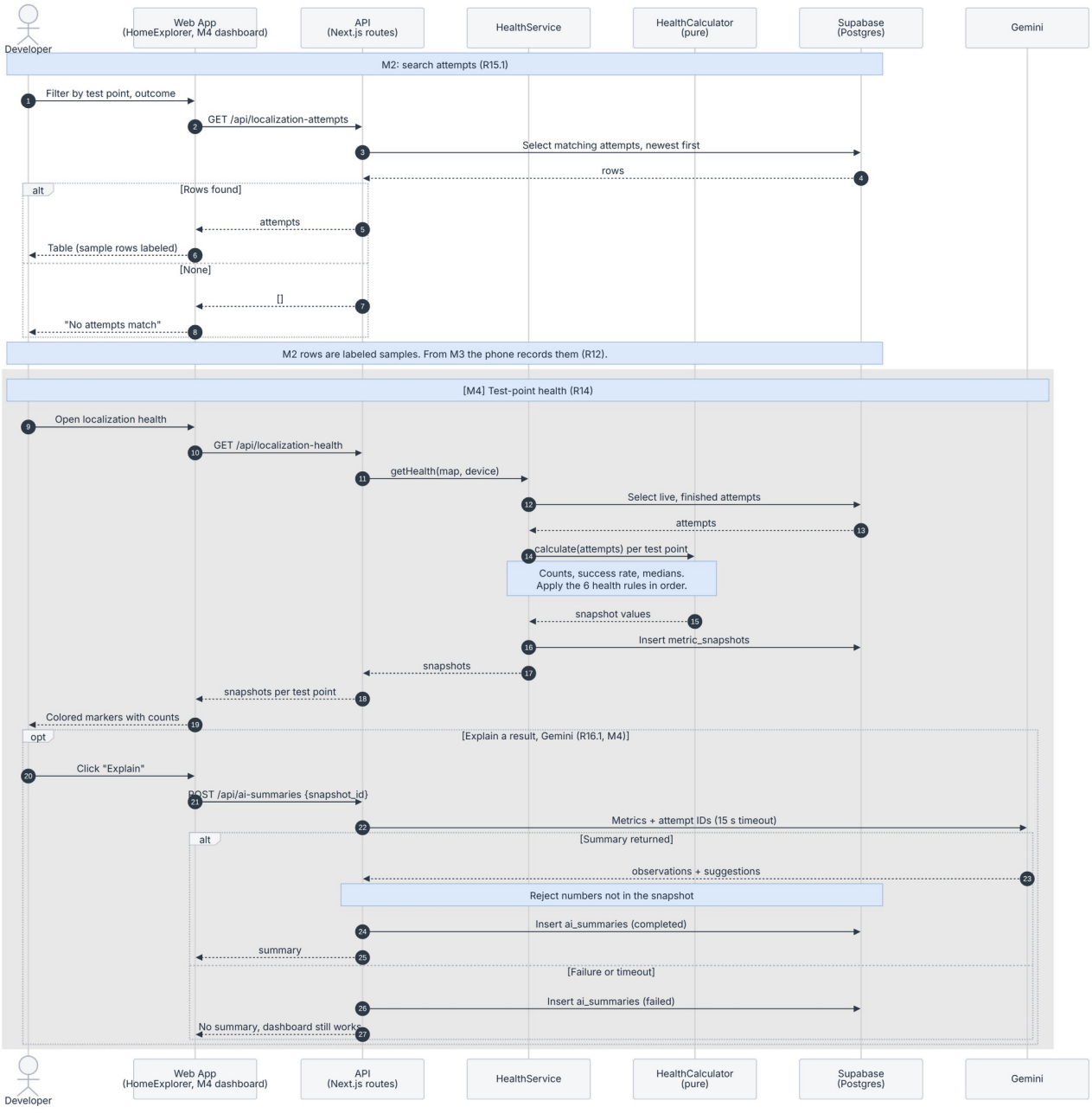


Figure 5.10: Attempt search and localization health

What to notice

- M2 search filters by test point and outcome, shows the newest first, labels sample rows, and shows “No attempts match” when nothing matches.

- Health uses only live, finished attempts from the same test point, device and map version; in_progress attempts are excluded. HealthService loads and saves; HealthCalculator only calculates.
- Health rules, in order, first match wins (Section 1.14): 1 gray, fewer than 10 attempts; 2 red, success below 60%; 3 red, at least 5 valid errors and median error above 1.5 m; 4 gray, fewer than 5 valid errors; 5 green, success 80% or more and median error 1 m or less; 6 yellow, all other cases. Counts are always shown with the color.
- This flow covers the colored markers (R14). The charts of R13 (M4) are not shown here.
- Gemini gets only aggregate metrics and attempt IDs. Numbers that do not match the snapshot are rejected, and the dashboard keeps working if Gemini fails or times out.

Flow 7: Stream the live phone position and follow the route

Purpose: A person walks with the iPhone while an observer watches /live. The dot moves on the map, the route starts from the node nearest the phone, and the page says when the person has arrived.

Requirements: R3.2, R3.3, R4.2, R6.2.1 (built early); R3.5, R6.5 (M2); R4.1 (partial); R6.2.2, R6.4 (M3).

Scope: [M2 partial] M2: pose upload, live dot, nearest-node start, route and arrival. M3: phone tracking states (R4.1), no route from a stale or lost pose (R6.4), start on the nearest path segment (R6.2.2).

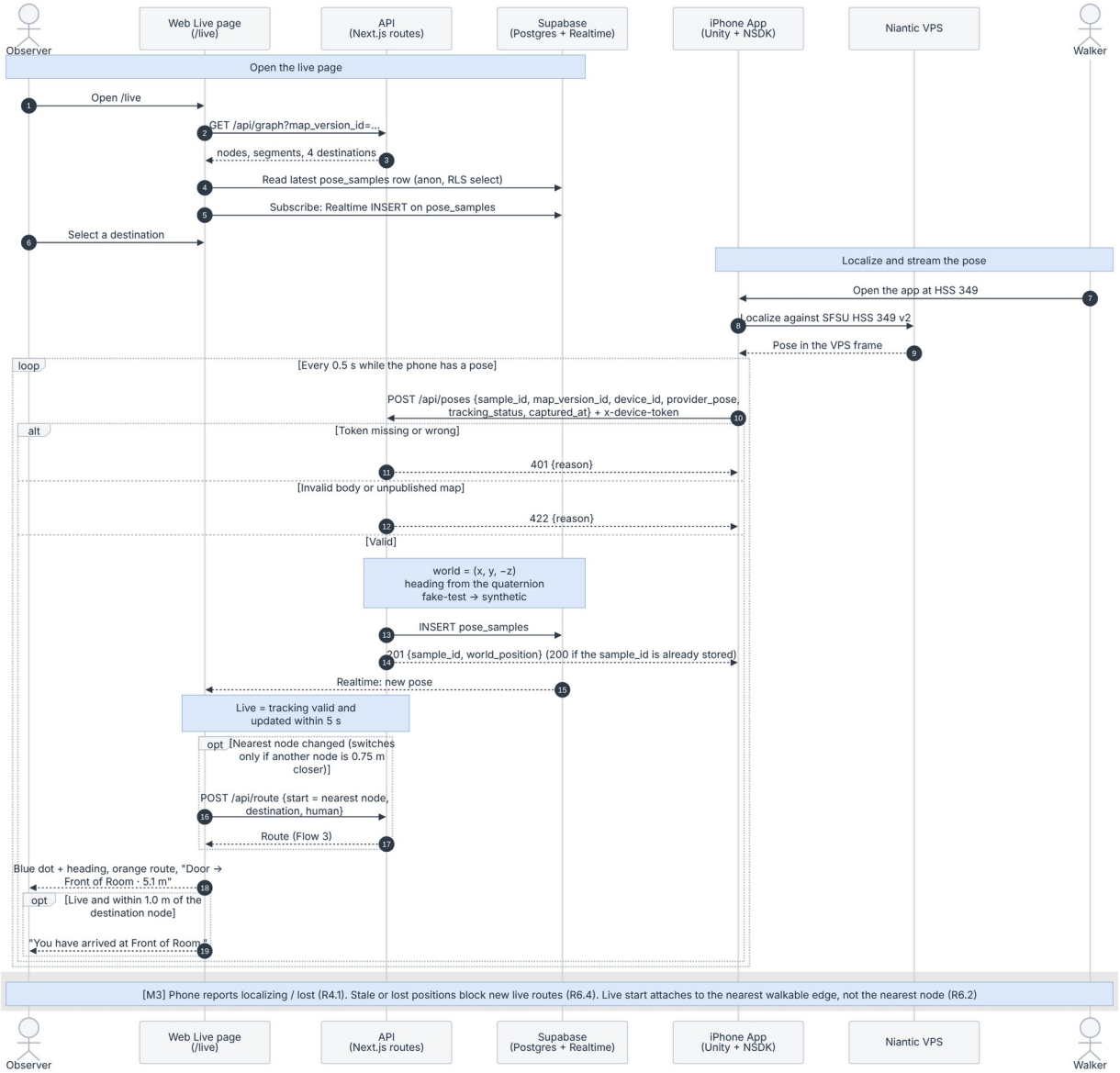


Figure 5.11: Live position, route and arrival

What to notice

- The phone sends the raw VPS pose every 0.5 s with a shared device token. The server flips Z into the world frame, computes the heading and stores both the raw and the converted pose in pose_samples. A repeated sample_id returns 200 and creates no duplicate.
- /live reads the latest pose once, then receives each new pose through Supabase Realtime. The position is live only when tracking is valid and the pose is less than 5 seconds old; otherwise the page shows “Not live” and the dot fades.
- The route start is the graph node nearest the phone (R6.2.1). It changes only when another node is at least 0.75 m closer, so the route does not flicker between two nodes. Each new start requests POST /api/route again (Flow 3).
- “You have arrived” appears only when the position is live and within 1.0 m of the destination node (R6.5). Choosing the same start and destination by hand shows “No route needed” instead, so old data never claims an arrival.
- Poses from the fake-test device (a pipeline test without localizing) are stored as synthetic and labeled “test pose” on /live (R3.5). In M2 they still move the dot and can start a route or trigger arrival; excluding synthetic poses from live navigation is planned for M3.
- In M2 the phone reports tracking as valid whenever it has a pose. Reporting localizing and lost (R4.1) and blocking routes from stale or lost positions (R6.4) follow in M3.

5.3 Traceability

Flow	Requirements	Main data classes	Built in M2	Later
1. Localize iPhone	R3.1; R3.4, R4.1, R12.1, R12.2	LocalizationAttempt, TestPoint, PoseSample, FrameTransform	Localize on the phone, show result (R3.1)	Attempt records, timeout, tracking states (M3)
2. View scanned area	R2.2, R2.3, R2.4, R15.3	MapVersion, VisualAsset, GraphNode, PathSegment, Destination	All	Active map from the site record, visual_assets table (M3)
3. Route	R5.1–R5.5, R6.1	Route, RouteStep, GraphNode, PathSegment, Destination	Manual-start route for a person or the robot, with distance	Revision check (409), stored routes (M3)
4. Close / reopen segment	R8.1–R8.3, R9.1, R9.3, R17.2	PathSegment, PathEvent, Route	Seeded availability only	Operator workflow (M3)
5. Simulated robot	R10.1–R10.5, R9.2, R18.1–R18.6	SimulatedAgent, SimulatedObstacle, AgentReport, PathEvent	Robot route profile only (Flow 3)	Start, move, replan (M3); pause, reset, obstacles (M4); reject (M5)
6. Attempts and health	R15.1; R12.1, R12.2; R14.1–R14.5, R16.1	LocalizationAttempt, MetricSnapshot, AiSummary	Search sample attempts (R15.1)	Recording (M3); health, AI summary (M4)
7. Live position	R3.2, R3.3, R3.5; R4.1, R4.2; R6.2.1, R6.2.2, R6.4, R6.5	PoseSample, GraphNode, Route	Pose upload, live dot, nearest-node start, route, arrival	Tracking states, stale/lost route block, segment attachment (M3)

The graph editor (R7.1–R7.2 in M4, R7.3 in M5) is not one of the seven flows. It follows the graph revision rule in decision 3 below.

5.4 Design decisions

1. **One routing engine for humans and robots.** Only the profile changes, so there is one algorithm to test. /viewer and /live both use it through POST /api/route.

2. **Optimistic concurrency using revision checks (M3).** Each change states the revision it expects, and so does each route request. The check and the write are one atomic conditional update, and a stale request gets 409. In M2 the route response already reports the revisions it used.
3. **Whole graph revisions.** A graph edit publishes a complete new graph revision (all nodes and segments) and switches current_graph_revision in one transaction. The old revision is kept, so stored routes and history still resolve. To be confirmed with the schema owner.
4. **Idempotent uploads that never go backwards.** One upsert with rules A–D (Flow 1): a missing record is created, retries change nothing, and a finished attempt never returns to in_progress. Failures are saved without a pose. Pose samples already work this way in M2: a repeated sample_id is not stored twice.
5. **Raw pose in, calculations on the server.** The phone sends the raw VPS pose. In M2 the server converts it and decides data_origin; from M3 it also calculates error and health.
6. **Pure logic separate from input and output.** Routing, pose origin and health rules have no database code, so they can be unit tested. Routing has 13 tests, and the full suite has 74.
7. **Check results before showing them.** Every route is validated against the database graph before it is sent, and arrival is shown only for a live, recent pose.
8. **One world frame per map.** For HSS 349 the world frame is the VPS map frame, so the scan, the graph nodes and the live poses line up without a stored transform. Other maps will use verified frame transforms (M3).
9. **AI explains numbers, never decides them.** The health color comes from fixed rules; Gemini output is checked against the snapshot and is optional.

5.5 Implementation status

Unit (Figures 5.3a, 5.3b)	Status on Oct 19, 2026	Where
RoutingEngine (findRoute), MinHeap	Merged into dev (PR #32). 13 Vitest tests pass.	app/frontend/lib/routing
GraphService, RouteService, LocalizationService (GET), SupabaseRepository, validation, routing adapter	Merged (PR #34) and deployed on Vercel.	app/frontend/app/api, app/frontend/lib/spatial/server
RoutePlanner, SceneViewer (/viewer)	Merged (PR #31). Real HSS 349 scan, database graph, POST /api/route and four destinations merged in PR #43 and released to main through PR #44 (Oct 19); deployed on Vercel and checked on the production URL.	app/frontend/components/viewer
HomeExplorer (home page list and search)	Merged (PR #39).	app/frontend/components/home
PoseService, LivePage (/live), PoseUploader	Merged (PR #35, #38, #40). Synthetic labeling of fake-test poses in PR #43, released through PR #44 and deployed (Oct 19).	app/frontend/app/api/poses, app/frontend/app/live, app/phone/live-position
iPhone app (Unity + NSDK)	Merged (PR #33). VPS localization in HSS 349 shown in class on Oct 13.	app/phone
Automated tests	74 tests pass: 61 API and logic tests, 13 routing tests.	npm test in app/frontend
All other units	Proposed design	—

PR numbers refer to github.com/ezerenm/SFSU-csc648-06-Fall26-team02.